

SET-1 — MASTER PRACTICAL GUIDE

Hospital Management System

Windows PC • Eclipse • Git/GitHub • Maven • Tomcat • Docker

Repository	https://github.com/ssvkotamraju/HMS.git
Coverage	28 individual task entries, each followed immediately by an answer
Exam strategy	Command-first, GUI-supported, with older-version fallbacks and examiner-proof checks

Use this as the exam-day walkthrough: open Git Bash/Command Prompt, Eclipse, Docker Desktop, and a browser side by side. For every task, perform the command, verify the result, and only then move on.

HOW TO USE THIS MASTER PDF

This guide is written for a Windows PC and is designed for practical-exam demonstration. Commands are the most version-tolerant method; GUI routes are included where Eclipse/Docker Desktop can be used. Exact menu labels can differ slightly between old and new releases, so use the command equivalent if a GUI label is missing.

Important source limitation: several exam questions refer to a “provided pom.xml”, but the uploaded question papers contain the questions, not the actual POM file. For those questions, this guide deliberately shows how to derive the exact value from the supplied POM instead of inventing coordinates, WAR names, or context paths.

1. Windows tool setup

Tool	What to verify	Windows command / GUI
Java JDK	Use a JDK, not only a JRE.	<code>java -version</code> <code>javac -version</code>
Maven	Maven available to the terminal, or Maven Wrapper exists.	<code>mvn -version</code> <code>mvnw.cmd -version</code>
Git	Git Bash/Command Prompt can run Git.	<code>git --version</code>
Eclipse	Maven integration (m2e) and Java support.	Window → Preferences → Java
Tomcat	Choose a Tomcat version compatible with the project.	Servers view → New → Apache Tomcat
Docker	Docker daemon/Desktop is running.	<code>docker version</code> Docker Desktop GUI

If `mvn` is not recognized

Either install/configure Maven and add its **bin** directory to Windows PATH, or check whether the repository contains **mvnw.cmd**. The wrapper is often the safest option for older student projects because it uses the project's expected Maven distribution.

```
where mvn
mvn -version
mvnw.cmd -version
```

Windows GUI: Control Panel / Settings → System → Advanced system settings → Environment Variables → create MAVEN_HOME=C:\apache-maven-3.x.x → edit Path → add %MAVEN_HOME%\bin → reopen terminal.

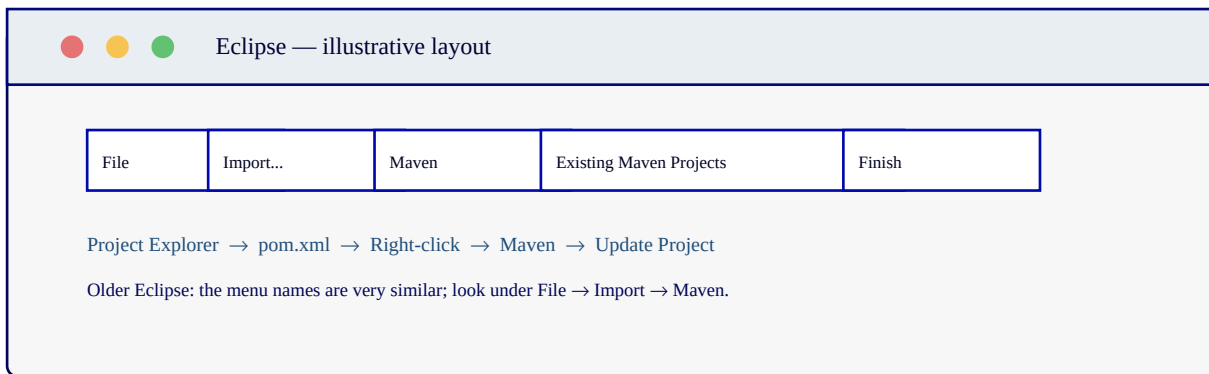
Java compatibility rule

Always run **mvn -version**, not only **java -version**, because Maven reports the Java home it actually uses. If an old project expects Java 8/11 and a newer JDK is installed, align the JDK used by Maven and Eclipse with the project's compiler settings. Do not blindly change the POM until you know the required source/target/release level.

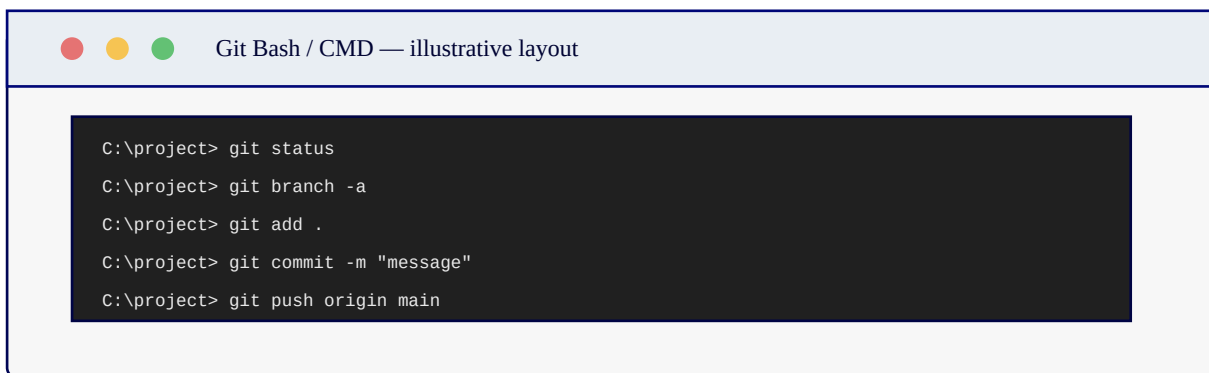
Tomcat compatibility warning

Older JSP/Servlet projects commonly use the **javax.servlet** API. Tomcat 10+ moved to the **jakarta.servlet** namespace. If the project imports `javax.servlet.*`, Tomcat 9 (or another compatible javax-based server) is usually the safer exam choice unless the code has already been migrated to Jakarta.

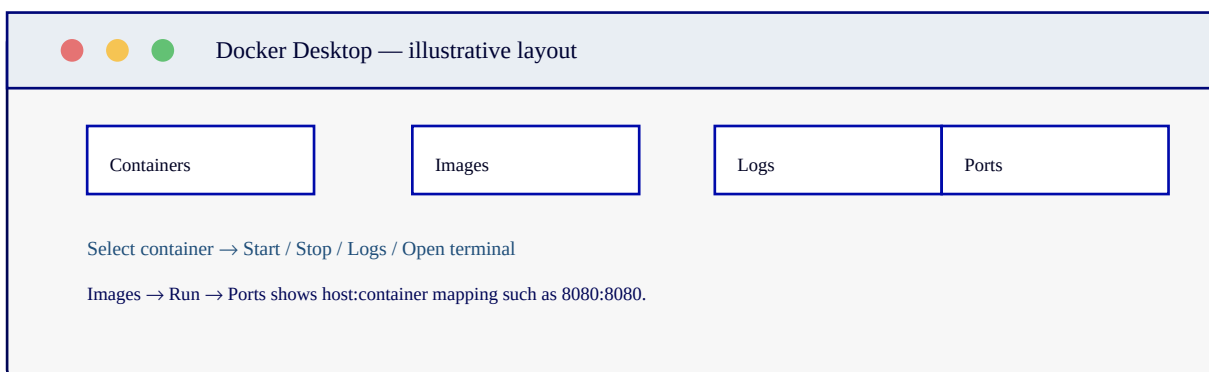
2. GUI routes you can demonstrate to the examiner



Eclipse Maven import: **File** → **Import** → **Maven** → **Existing Maven Projects** → Browse to the cloned folder → select pom.xml → Finish. Then right-click the project → Maven → Update Project. For older Eclipse releases, the wording is almost identical.



Git tasks are best demonstrated in Git Bash because every advanced command in these papers is available there. GitHub Desktop/Eclipse EGit can perform many common operations, but rebase, reflog, patch, and detailed recovery are much easier to prove from Git Bash.



Docker Desktop is useful for showing Images, Containers, Logs, and port mappings. If the lab machine has an older Docker Desktop layout, the same objects may appear under older tabs/menu names. The CLI remains the most reliable fallback.

3. Universal exam command sheet

Area	Commands
Git basics	<code>git clone URL</code> <code>cd project</code> <code>git status</code> <code>git remote -v</code> <code>git branch -a</code>
Branching	<code>git switch -c feature/name</code> # older Git: <code>git checkout -b feature/name</code>
Inspect changes	<code>git diff</code> <code>git diff --cached</code> <code>git status</code>
Commit	<code>git add file</code> <code>git commit -m "message"</code> <code>git log -1 --oneline</code>
Remote sync	<code>git fetch origin</code> <code>git merge origin/main</code> <code>git pull origin main</code>
Rebase	<code>git fetch origin</code> <code>git rebase origin/main</code> <code>git rebase --abort</code>
Undo safely	<code>git revert <hash></code> <code>git reset --soft HEAD~1</code> <code>git reflog</code>
Stash	<code>git stash push -m "work"</code> <code>git stash list</code> <code>git stash pop</code>
Compare/history	<code>git diff main feature -- file</code> <code>git log --oneline --graph --decorate --all</code>
Maven	<code>mvn -version</code> <code>mvn clean package</code> <code>mvn dependency:tree</code> <code>mvn dependency:analyze</code> <code>mvn clean package -X</code>
Tests	<code>mvn test</code> <code>mvn -Dtest=ClassName test</code> <code>mvn clean package -DskipTests</code>
Docker	<code>docker build -t app:v1 .</code> <code>docker run -d --name app -p 8080:8080 app:v1</code> <code>docker ps</code> <code>docker logs app</code>
Docker Hub	<code>docker login</code> <code>docker tag app:v1 user/app:v1</code> <code>docker push user/app:v1</code> <code>docker logout</code>

4. Universal troubleshooting decision tree

Maven failure: read the first meaningful [ERROR] → identify the phase/goal → inspect the POM coordinates/configuration → run `dependency:tree/effective-pom` → check JDK with `mvn -version` → clean → rebuild with `-X`.

Browser failure: prove Maven created the artifact → prove the server started → prove the port is listening → prove the context path → read server logs → then inspect JSP/application code.

Docker failure: prove Docker daemon → prove Dockerfile/context → prove image exists → prove container exists → inspect `docker ps -a` → inspect docker logs → inspect port mapping.

5. COMPLETE TASK-BY-TASK QUESTION → ANSWER GUIDE

The task wording below is kept close to the uploaded question paper while being shortened where a long scenario is repeated. Every numbered/sub-numbered practical requirement is represented. Where the source asks for an exact value that depends on an unseen POM, the answer explicitly tells you how to derive it.

Q1 — TASK / QUESTION

Hospital Management System scenario: identify the core requirements that the system should satisfy.

ANSWER / PROCEDURE

Translate the scenario into functional and non-functional requirements, actors, core use cases, and deployment needs. For a web hospital/ticketing/recruitment system, include authentication, role-based access, CRUD workflows, reports, security, usability, and concurrent-user performance.

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

2(a) — TASK / QUESTION

Clone HMS and import the latest source into Eclipse. Which Git commands and Eclipse steps should be followed?

ANSWER / PROCEDURE

1) Install Git for Windows. Open Git Bash. 2) Move to the folder where you want the project. 3) Clone the repository. 4) Enter the folder and verify the remote. 5) In Eclipse use File → Import → Maven → Existing Maven Projects (older Eclipse: File → Import → Maven → Existing Maven Projects), select the folder containing pom.xml, Finish, then Project → Clean if necessary.

Commands / configuration

```
git clone https://github.com/ssvкотamraju/HMS.git
cd HMS
git status
git remote -v
```

GUI route: Eclipse → File → Import → Maven → Existing Maven Projects; for an already imported project use Right-click → Maven → Update Project.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(b) — TASK / QUESTION

`mvn clean install` says `mvn` is not recognized`. Give the cause and Windows configuration fix.

ANSWER / PROCEDURE

`mvn` is not recognized because Maven is not installed or its `bin` directory is not on PATH. On Windows: download/extract Maven (for example `C:\apache-maven-3.x.x`), create `MAVEN\_HOME` pointing to that folder, and add `%MAVEN\_HOME%\bin` to Path. Close and reopen the terminal, then verify with `mvn -version`. Eclipse can also use its embedded Maven, but the terminal command needs a system Maven or a wrapper (`mvnw`).

Commands / configuration

```
where mvn
echo %MAVEN_HOME%
mvn -version
```

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

2(c) — TASK / QUESTION

Eclipse cannot resolve one or more Maven dependencies. How do you identify and fix the problem?

ANSWER / PROCEDURE

In Eclipse, right-click the project → Maven → Update Project... (older Eclipse/m2e versions use the same Maven menu). Check Force Update of Snapshots/Releases when a stale dependency is suspected, then Project → Clean.

GUI route: Eclipse → File → Import → Maven → Existing Maven Projects; for an already imported project use Right-click → Maven → Update Project.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

2(d) — TASK / QUESTION

`mvn clean install` fails during compilation. How do you determine the cause?

ANSWER / PROCEDURE

Run Maven with debug logging and preserve the first meaningful error. Do not start by changing random POM values. Check the first `[ERROR]`, the goal/phase named, and the `Caused by:` chain.

Commands / configuration

```
mvn clean package -X
mvn clean package -e
mvn -version
```

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

2(e) — TASK / QUESTION

The project was developed with a different Java version. How do you troubleshoot and resolve compatibility?

ANSWER / PROCEDURE

Check ``<maven.compiler.source>``/``<maven.compiler.target>`` or the `maven-compiler-plugin` configuration. Use a compiler level supported by the installed JDK and by the codebase. In older projects you will often see `1.8`; modern Maven Compiler Plugin versions also support a `release` setting.

Commands / configuration

```
mvn -version
mvn help:effective-pom
mvn clean compile
```

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

2(f) — TASK / QUESTION

Tests fail because an external service is unavailable, but you need the package now. How do you skip tests and what are the risks?

ANSWER / PROCEDURE

For a temporary package when tests cannot run, use `-DskipTests`. It compiles tests but skips running them. `-Dmaven.test.skip=true` skips compiling and running tests and is more aggressive. Explain the risk: the artifact can be produced without current test evidence.

Commands / configuration

```
mvn clean package -DskipTests
mvn clean package -Dmaven.test.skip=true
```

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

2(g) — TASK / QUESTION

Two libraries depend on different versions of the same library. How do you identify and resolve the Maven conflict?

ANSWER / PROCEDURE

Use the dependency tree to see which paths bring different versions. Maven normally selects the nearest definition in the dependency graph; a dependencyManagement entry or explicit direct dependency can be used to pin the intended version. Exclusions remove an unwanted transitive path.

Commands / configuration

```
mvn dependency:tree -Dverbose
mvn help:effective-pom
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

2(h) — TASK / QUESTION

The application worked before a pom.xml dependency change and behaves unexpectedly afterward. How do you troubleshoot?

ANSWER / PROCEDURE

A clean build removes the old `target` directory, so stale classes, old generated files, and previous artifacts cannot hide a POM/configuration change.

Commands / configuration

```
mvn clean
mvn clean package
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

2(i) — TASK / QUESTION

Maven says BUILD SUCCESS but the HMS application does not load in the browser. What checks do you perform?

ANSWER / PROCEDURE

A successful Maven build only proves the build lifecycle completed. Verify the WAR/JAR exists, start the server, confirm the server log says it started, confirm the expected port is listening, then open the correct context path. Check browser/server logs for 404/500 errors.

Commands / configuration

```
dir target
netstat -ano | findstr :8080
tasklist | findstr /I java
```

GUI route: Eclipse → Window → Show View → Servers → add/start Tomcat; then open the URL in a browser and check the Servers/Console view for startup errors.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

2(j) — TASK / QUESTION

A required Maven build plugin cannot be found/executed. Which pom.xml sections do you inspect?

ANSWER / PROCEDURE

For a plugin execution failure, inspect ``<build><plugins>`` and the plugin's ``<groupId>``, ``<artifactId>``, ``<version>``, and ``<configuration>``. ``<pluginManagement>`` only supplies defaults; it does not activate a plugin by itself. If the command uses ``tomcat7:run``, the plugin normally needs the Apache Tomcat Maven Plugin coordinates.

Commands / configuration

```
mvn help:effective-pom
mvn tomcat7:run
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

3(a) — TASK / QUESTION

Initialize Git in a normal HMS folder and create the first commit.

ANSWER / PROCEDURE

1) Install Git for Windows. Open Git Bash. 2) Move to the folder where you want the project. 3) Clone the repository. 4) Enter the folder and verify the remote. 5) In Eclipse use File → Import → Maven → Existing Maven Projects (older Eclipse: File → Import → Maven → Existing Maven Projects), select the folder containing pom.xml, Finish, then Project → Clean if necessary.

Commands / configuration

```
git clone https://github.com/ssvkotamraju/HMS.git
cd HMS
git status
git remote -v
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

3(b) — TASK / QUESTION

Add mobile-number/password-validation changes to register.jsp using a safe Git workflow.

ANSWER / PROCEDURE

Create and switch in one operation. This is the modern command; on older Git use the checkout equivalent.

Commands / configuration

```
git switch -c feature/registration-validation
git checkout -b feature/registration-validation
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

3(c) — TASK / QUESTION

Inspect an unwanted modification to register.jsp before committing.

ANSWER / PROCEDURE

Use `git status` first. It separates modified-but-unstaged files, staged files, and untracked files. For exact line-level edits use `git diff`; for staged edits use `git diff --cached`.

Commands / configuration

```
git status
git diff
git diff --cached
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

3(d) — TASK / QUESTION

Stage modified register.jsp and a new CSS file for commit.

ANSWER / PROCEDURE

Use `git status` first. It separates modified-but-unstaged files, staged files, and untracked files. For exact line-level edits use `git diff`; for staged edits use `git diff --cached`.

Commands / configuration

```
git status
git diff
git diff --cached
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

3(e) — TASK / QUESTION

Undo a committed change that breaks registration while preserving history.

ANSWER / PROCEDURE

Use `git revert` when the commit has already been recorded/shared and you want to preserve history. Git creates a new commit that reverses the old commit.

Commands / configuration

```
git log --oneline
git revert <commit-hash>
git log --oneline -2
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

3(f) — TASK / QUESTION

Unstage a configuration file without deleting it.

ANSWER / PROCEDURE

Remove the file from Git tracking but leave the physical file on disk. Then add the pattern to `.gitignore` so it is not accidentally committed again.

Commands / configuration

```
git rm --cached "configuration-file"
echo db-config.env >> .gitignore
git add .gitignore
git commit -m "Stop tracking local configuration"
```

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

3(g) — TASK / QUESTION

You are halfway through register.jsp and must switch branches for an urgent fix. What do you do?

ANSWER / PROCEDURE

Stash saves uncommitted work without creating a commit. Give the stash a name so it is easy to recognize.

Commands / configuration

```
git stash push -m "unfinished work"
git switch <emergency-branch>
git stash list
git stash pop
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

3(h) — TASK / QUESTION

A feature branch is behind main and the team wants a clean history. What do you do?

ANSWER / PROCEDURE

Fetch first, then rebase your feature branch onto the updated main branch. This rewrites the feature branch commit IDs, so do it before sharing the branch or be prepared to use `--force-with-lease` when pushing.

Commands / configuration

```
git fetch origin
git switch feature/your-branch
git rebase origin/main
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

4(i) — TASK / QUESTION

Create a Dockerfile and build an HMS image for Docker Hub.

ANSWER / PROCEDURE

For a JSP/Servlet WAR, use Tomcat as the runtime. The WAR is copied into Tomcat's `webapps` directory. A multi-stage build is cleaner because Maven is used only in the build stage.

Commands / configuration

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM tomcat:9-jdk17
RUN rm -rf /usr/local/tomcat/webapps/ROOT
COPY --from=build /app/target/*.war /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

3(j) — TASK / QUESTION

Push tested HMS changes to GitHub.

ANSWER / PROCEDURE

Push the branch and then compare the local commit ID with the remote-tracking branch after fetching. Equal hashes mean both point to the same commit.

Commands / configuration

```
git push -u origin main
git fetch origin
git rev-parse HEAD
git rev-parse origin/main
git status
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

4(b) — TASK / QUESTION

What important Dockerfile instructions are needed to containerize HMS?

ANSWER / PROCEDURE

For a JSP/Servlet WAR, use Tomcat as the runtime. The WAR is copied into Tomcat's `webapps` directory. A multi-stage build is cleaner because Maven is used only in the build stage.

Commands / configuration

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM tomcat:9-jdk17
RUN rm -rf /usr/local/tomcat/webapps/ROOT
COPY --from=build /app/target/*.war /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

GUI route: Eclipse → File → Import → Maven → Existing Maven Projects; for an already imported project use Right-click → Maven → Update Project.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

4(ii) — TASK / QUESTION

Run the HMS image and expose port 8080; verify localhost:8080.

ANSWER / PROCEDURE

Start detached, map host 8080 to container 8080, and name the container so it is easy to demonstrate.

Commands / configuration

```
docker run -d --name hms-container -p 8080:8080 HMS:v1
docker ps
docker port hms-container
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

4(a) — TASK / QUESTION

Explain how Docker gives every HMS developer a consistent Java/server environment.

ANSWER / PROCEDURE

Before writing the Dockerfile, determine whether the project produces a JAR or WAR, the Java version, the server/runtime (Tomcat for JSP/Servlet apps), the artifact name under `target`, the application port, and the startup command. Then choose the matching base image and copy the artifact into the correct runtime location.

Commands / configuration

```
mvn package
dir target
docker version
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

4(b) — TASK / QUESTION

What important Dockerfile instructions are needed to containerize HMS?

ANSWER / PROCEDURE

If the question explicitly asks for `openjdk:17-jdk-slim` and a JAR, use a simple runtime image. The JAR must exist before `docker build`. For older Docker engines the syntax below is broadly compatible.

Commands / configuration

```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/HMS.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

GUI route: Eclipse → File → Import → Maven → Existing Maven Projects; for an already imported project use Right-click → Maven → Update Project.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

4(c) — TASK / QUESTION

docker build fails. How do you inspect the Dockerfile and build output?

ANSWER / PROCEDURE

Read the build error from the first failing instruction, check the build context and file names, confirm the Dockerfile is in the current directory, and rebuild without guessing. For runtime failures, inspect `docker ps -a` and `docker logs`.

Commands / configuration

```
docker build -t app:v1 .
docker ps -a
docker logs <container>
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

4(d) — TASK / QUESTION

The app listens on 8080 in the container but localhost:8080 cannot reach it. Explain port mapping.

ANSWER / PROCEDURE

A container port is not automatically published to the Windows host. `-p 8080:8080` means `HOST_PORT:CONTAINER_PORT`. Verify the mapping with `docker port` and then browse to localhost:8080.

Commands / configuration

```
docker run -d --name app -p 8080:8080 app:v1
docker port app
netstat -ano | findstr :8080
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

4(e) — TASK / QUESTION

Tag and push the local HMS image to Docker Hub.

ANSWER / PROCEDURE

Authenticate, tag the local image with your Docker Hub namespace, push it, then verify on Docker Hub or with `docker pull` from another machine.

Commands / configuration

```
docker login
docker tag HMS:v1 <dockerhub-username>/HMS:v1
docker push <dockerhub-username>/HMS:v1
docker pull <dockerhub-username>/HMS:v1
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

6. EXAMINER DEMONSTRATION CHECKLIST

- Open the project folder and prove the repository/remote is correct.
- Open pom.xml and point to groupId, artifactId, version, packaging, dependencies and plugins when asked.
- Run mvn -version and, if required, java -version/javac -version.
- Build with mvn clean package (or the exact lifecycle command in the question).
- Show target/ and the generated WAR/JAR.
- In Eclipse, use Maven → Update Project when dependencies are stale.
- For Git, show status/diff before commit and log after commit.
- For rebase/merge questions, show the branch name and final clean status.
- For Docker, show docker images, docker ps -a, docker logs and port mapping as appropriate.
- Open http://localhost:8080 only after proving the server/container is running.
- For Docker Hub, show tag → login → push → repository verification.

7. OLD-VERSION FALLBACKS

If you see...	Use this fallback
Old Git without `git switch`	Use `git checkout -b branch` / `git checkout branch`.
Old Eclipse	Look under File → Import → Maven and Window → Show View → Servers; labels vary slightly.
Old Maven project	Check for `mvnw.cmd`; otherwise use a Maven version compatible with the project/JDK.
Old Tomcat project using javax.servlet	Prefer a javax-compatible Tomcat release (commonly Tomcat 9) unless the project is migrated to Jakarta.
Docker Desktop GUI differs	Use the CLI; the core objects remain images, containers, logs, and port mappings.
PowerShell command differs	Git Bash is the simplest universal shell for the Git commands in these papers.

Official references used for the practical workflow

Git for Windows: <https://git-scm.com/install/windows>

Eclipse documentation: <https://www.eclipse.org/documentation/>

Maven Compiler Plugin: <https://maven.apache.org/plugins/maven-compiler-plugin/>

Maven Dependency Plugin: <https://maven.apache.org/plugins/maven-dependency-plugin/>

Docker Desktop for Windows: <https://docs.docker.com/desktop/setup/install/windows-install/>